

Permissions, models, and building with Claude Code

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN Session 3 of 12 · English notes

The last teaching block of 6 August was how you **steer** Claude Code before you let it write an app: permission modes, `/model`, `/config`, where settings files live, and `/init` → **CLAUDE.md** as persistent project law. Later modules go deeper on modes and agents; this chapter is the Foundation control panel.

Permission modes (as labelled in class)

The Code UI grouped how freely the agent may use tools:

Mode (classroom language)	What you are choosing	When Vijay would use it
Ask / manual permission per tool	Each sensitive tool call waits for you	Untrusted repo, first day on a client codebase
Auto-edit in the workspace	Edits proceed without a click each time	A folder you already trusted at CLI start
Plan first	Emit a sequenced plan; you review/customise before execute	Brownfield, expensive refactors, exam-style “don’t just run”
Bypass / don’t wait	Skip permission prompts and run	Only on throwaway sandboxes — not UST client trees

First CLI launch already asked “do you trust this git folder?” That consent is the root of auto-edit. Allow/deny lists also persist in settings JSON (deny reading secrets/API keys was Lalit’s example). You can ask Claude to rewrite its own settings — it still needs permission to touch those files.

Effort and `/model`

Effort is how hard the selected model should think—not a separate product. `/model` listed what the org subscription enabled (in class: Sonnet, Opus, Haiku, plus names such as Fable). Typing `opus` took the latest Opus SKU he had (he cited a 4.8-class line); you may also pass a full model ID from the docs. Switching to a 1M Sonnet changes the *window*, not automatically the quality of a one-line answer. Hussain’s question: you do not have to memorise every dated ID if the short name resolves to current.

PICKER VERSUS DOCUMENTATION

Sonnet “5”, Opus “4.8”, Fable, 272K vs 1M were 6 August 2026 UI. Exam written answers should use IDs and capabilities from [the models page](#).

Slash config and settings.json layers

`/config` shows key/value pairs (editor backend: Git Bash vs PowerShell vs vim; UI language; agent toggles; auto-compact). Values are stored under a `.claude` directory (ASR: “dot clod”) as `settings.json` at more than one layer:

1. **User level** — under the OS profile; applies to all projects for that person. Creating auto-compact from `/config` often lands here first.
2. **Project level** — `.claude/settings.json` in the git repo (shareable conventions).
3. **Folder / local override** — `settings.local.json` to override project settings on this machine without committing secrets.

Precedence he taught: if all three exist, **folder/local wins**; else project; else user. Missing keys inherit from the layer above. This JSON is Claude Code’s own switches—not your ESLint rules. Clean-code standards, library pins, and “never read `.env`” as *behaviour* belong in **CLAUDE.md** (memory/instructions), which is a different file from `settings.json`.

CLAUDE.md via /init

`/init` tries to generate persistent project instructions. On an empty or unclear tree it may refuse until there is structure—you can create the markdown yourself. This is the same family of file Cowork auto-created in session 1. Ground rules (OpenAPI-only for frontend, no client PII in prompts, package manager) go here so every session does not re-explain the repo. Do not paste entire vendor manuals (session 1 token warning).

Building: you still owe a prompt and a review

With permissions, model, and CLAUDE.md in place, you can ask for an end-to-end app. Vijay’s promise: Code will plan, use tools, and verify. Your job on CCDV-F is to know *which knobs you turned* and what still needs a human (secrets, production deploy, client data). Later sessions generate without typing the app by hand; the exam will still ask why you chose plan-mode, a smaller model, or a new session.

Checklist before you “just generate”

- Trust dialog and permission mode match the sensitivity of the tree.
- `/model` matches cost vs window needs.
- Auto-compact on if you expect a long agentic loop.
- CLAUDE.md states stack and prohibitions; `settings.json` states product toggles.
- New session for a new feature so yesterday’s failed experiment is not in context.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Permissions, CLI, IDE, CLAUDE.md as currently documented.

- **Models**

<https://docs.claude.com/en/docs/about-claude/models>

IDs to pass to /model.

- **Extended thinking**

<https://docs.claude.com/en/docs/build-with-claude/extended-thinking>

What “effort / thinking” maps to in the API world.

- **Guardrails**

<https://docs.claude.com/en/docs/test-and-evaluate/strengthen-guardrails>

Enterprise counterpart to deny-lists in settings.json.

- **Usage policy**

<https://www.anthropic.com/legal/usage-policy>

What must never land in a trusted workspace prompt.

- **Structured outputs**

<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

On the developer Foundation skill list alongside Code.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.